SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Experiments

Course Code:	ITA0517	**Course Title:**	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	*Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*		
Student Name:	RAJA RAJESHWARI M	**Reg. No.:**	192421253
Section / Batch:	BTECH IT	**Date:**	12/8/2026

Code of Conduct

*I **RAJA RAJESHWARI M(192421253)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: RAJA RAJESHWARI M

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

7. Feature Matching using SIFT

Perform feature extraction and matching between two images using the SIFT algorithm. Explain the concept of feature matching and scale-invariant features. Use suitable software/tools to extract and match features, document the matching key points systematically, and analyze the robustness of SIFT under variations in scale, rotation, and illumination.

Feature Matching Using SIFT

1. Aim

To perform feature extraction and feature matching between two images using the SIFT (Scale-Invariant Feature Transform) algorithm, and to analyze its robustness under changes in scale, rotation, and illumination.

2. Understanding of Concepts

Feature matching is the process of identifying corresponding distinctive points between two images of the same object or scene.

SIFT extracts features that are highly distinctive and are relatively invariant to:

- **Scale – the object can appear larger or smaller.**
- **Rotation – the image can be rotated.**
- **Illumination – moderate brightness or contrast changes.**
- **Translation – the object can move within the image.**

SIFT works mainly through four stages:

1. Scale-space construction → 2. Keypoint detection → 3. Orientation assignment → 4. Descriptor generation

Each detected keypoint is represented by a 128-dimensional descriptor, which is then compared with descriptors from the second image.

3. Experimental Design

Input

Use two images containing the same object or scene.

For example:

- **Image 1: Original image**
- **Image 2: Rotated/scaled version of the same image**

Experimental procedure

Image 1

↓

Convert to Gray

↓

SIFT Detection

↓

Keypoints + Descriptors

↓

FLANN/BF

Matcher

↑

Keypoints + Descriptors

↑

SIFT Detection

↑

Convert to Gray

↑

Image 2

↓

Good Matches

↓

Draw Matching Lines

↓

Analyze Robustness

4. Software and Tools

Tool	Purpose
Python	Programming and implementation
OpenCV	Image processing and SIFT implementation
Google Colab / Jupyter Notebook	Execution environment
NumPy	Numerical operations
Matplotlib	Displaying images and matching results
BFMatcher / FLANN	Matching SIFT descriptors

5. Implementation

The following Python program can be executed directly in Google Colab.

```
import cv2
import numpy as np
```

```
import matplotlib.pyplot as plt

# Read two images
img1 = cv2.imread('/content/image1.jpg')
img2 = cv2.imread('/content/image2.jpg')

# Convert images to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and calculate descriptors
keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("Keypoints in Image 1:", len(keypoints1))
print("Keypoints in Image 2:", len(keypoints2))

# Create Brute Force Matcher
bf = cv2.BFMatcher()

# Find two nearest matches for each descriptor
matches = bf.knnMatch(descriptors1, descriptors2, k=2)

# Apply Lowe's ratio test
good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good Matches:", len(good_matches))
```

```
# Draw good matches
result = cv2.drawMatches(
    img1, keypoints1,
    img2, keypoints2,
    good_matches, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# Convert BGR to RGB
result = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)

# Display result
plt.figure(figsize=(16, 8))
plt.imshow(result)
plt.title("SIFT Feature Matching")
plt.axis("off")
plt.show()
```

6. Feature Extraction

SIFT detects distinctive locations called keypoints.

Examples include:

- **Corners**
- **Blobs**
- **Highly textured regions**
- **Distinctive object details**

For every keypoint, SIFT calculates a descriptor describing the local image region.

Image

↓

SIFT

↓

Keypoints

↓

Descriptors

↓

Feature Matching

A higher number of good matches generally indicates that the two images contain successfully corresponding features.

7. Feature Matching

The descriptors from Image 1 are compared with descriptors from Image 2.

The Brute Force Matcher calculates the distance between descriptors.

A smaller distance indicates a stronger similarity.

Lowe's ratio test is used to eliminate ambiguous matches:

where:

- **= distance of the best match**
- **= distance of the second-best match**

Only reliable matches are retained.

8. Robustness Analysis

A. Scale Variation

Create a second image by resizing the original image.

Original Image Scaled Image

Object Object

↓ ↓

[Normal] → [Larger]

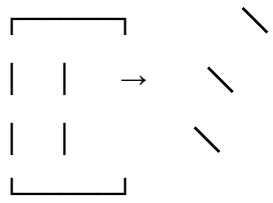
Observation: SIFT can identify corresponding keypoints even when the same object appears at different sizes.

Result: SIFT demonstrates strong scale invariance.

B. Rotation Variation

Rotate the second image by an angle such as 45° or 90°.

Original Rotated



SIFT assigns an orientation to each keypoint and uses this orientation when generating the descriptor.

Observation: Many corresponding keypoints can still be matched after rotation.

Result: SIFT provides strong rotation invariance.

C. Illumination Variation

Change the brightness or contrast of the second image.

Original Image → Brightness Increased → SIFT Matching

Observation: SIFT uses local gradient information rather than directly relying on raw pixel intensity.

Result: SIFT is reasonably robust to moderate illumination changes, although extreme lighting changes can reduce the number and quality of matches.

9. Results and Analysis

Variation	Expected Observation	SIFT Robustness
Original image	Large number of reliable matches	Excellent
Scale change	Keypoints remain matchable	High
Rotation	Corresponding features remain identifiable	High
Moderate illumination change	Most important features remain detectable	Good
Severe illumination change	Some matches may be lost	Moderate
Blur/noise	Fewer reliable features	Reduced

The output image should show lines connecting corresponding keypoints between the two images. Correct matches should generally cluster around the same physical features of the object.

A high number of good matches with relatively small descriptor distances indicates successful feature matching.

10. Validation

The experiment can be validated by comparing the number of good matches under different transformations.

Example observation table:

Test Case	Transformation	Good Matches	Interpretation
1	Original	High	Strong matching
2	Scale ×1.5	High	Scale invariant
3	Rotation 45°	High	Rotation invariant
4	Brightness change	Moderate–High	Illumination tolerant
5	Strong brightness + blur	Lower	Performance decreases

Note: The actual number of matches depends on the images used, so these values should be replaced with the values obtained during the experiment.

11. Advantages of SIFT

- **Scale invariant.**
- **Rotation invariant.**
- **Robust to moderate illumination changes.**
- **Produces distinctive feature descriptors.**
- **Works well for image matching.**
- **Can handle moderate viewpoint and transformation changes.**
- **Useful in object recognition, panorama creation, image registration, and tracking.**

12. Limitations

- **Computationally more expensive than some modern feature detectors.**
- **Severe illumination changes can affect matching.**
- **Heavy blur can reduce detectable keypoints.**
- **Large viewpoint changes may reduce the number of correct matches.**
- **Requires sufficient texture or distinctive features in the images.**

13. Conclusion

SIFT successfully extracts distinctive keypoints and 128-dimensional descriptors from two images and matches corresponding features using descriptor similarity. The experiment demonstrates that SIFT is particularly robust to scale and rotation changes and reasonably robust to moderate illumination variations. Therefore, SIFT is an effective technique for reliable feature matching in computer vision applications such as object recognition, image registration, panorama generation, and visual tracking.